

Universidad San Carlos de Guatemala
Facultad de ingeniería.
Ingeniería en ciencias y sistemas



FIUSAC
FACULTAD DE INGENIERÍA
UNIVERSIDAD DE SAN CARLOS DE GUATEMALA



Práctica:

Orquestación avanzada de microservicios en Kubernetes con Helm

PONDERACIÓN: 5 pts

Tiempo estimado: 15 hrs

Índice

1. MARCO FORMATIVO	P1
1.1. Valor	P1
1.2. Competencia(s)	P1
1.3. Habilidad(es) blandas a formar	P1
2. Resultado del Aprendizaje	P2
2.1. Objetivo SMART	P2
3. Enunciado de la Práctica	P3
3.1 Descripción del problema a resolver	P3
3.2 Alcance de la práctica	P4
3.3 Requerimientos técnicos	P5
4. Entregables	P6
5. Material de apoyo	P7
6. Recursos y herramientas a utilizar	P7
7. Cronograma	P8
8. Rúbrica de Calificación	P8
8.1 Requisitos para optar a la calificación	P8
8.2 Resumen de Puntuaciones	P9
Detalle de la Calificación	P10
Comentarios Generales	P11

1. MARCO FORMATIVO

1.1. Valor

Nombre del valor	¿Cómo se aplica en tu laboratorio?
Responsabilidad Profesional y Ética en el Desarrollo de Software	El estudiante deberá empaquetar, versionar y desplegar una plataforma de microservicios cumpliendo estándares de calidad, principios de configuración externalizada, manejo seguro de credenciales y documentación técnica reproducible. Deberá respetar las normas de originalidad y trabajo individual, así como declarar de forma transparente cualquier apoyo de herramientas de inteligencia artificial.

1.2. Competencia(s)

Con la elaboración de esta práctica usted adquirirá las siguientes competencias:

2. **Competencia General:** Aplicar arquitecturas modernas de desarrollo de software para construir sistemas escalables, resilientes, seguros y desacoplados, utilizando tecnologías de orquestación vigentes en la industria.
3. **Competencia Específica (Técnica):** Empaquetar una plataforma de microservicios como un chart de Helm parametrizable por ambiente, incorporando comunicación asíncrona mediante un broker de mensajería, persistencia con almacenamiento durable, aislamiento de red y control de acceso basado en roles dentro de un clúster de Kubernetes.

3.1. Habilidad(es) blandas a formar

La práctica le permitirá desarrollar las siguientes habilidades:

- **Resolución de problemas:** Capacidad para diagnosticar fallos en sistemas distribuidos (pods en CrashLoopBackOff, probes fallidas, colas saturadas, políticas de red mal definidas) utilizando evidencia y no ensayo y error.
- **Pensamiento sistémico:** Comprender cómo un cambio en un valor de configuración se propaga a través de plantillas, releases y componentes interdependientes.
- **Organización y Gestión del Tiempo:** Planificar la carga de trabajo en las 15 horas estimadas, priorizando los componentes críticos antes que los opcionales.
- **Comunicación técnica:** Documentar decisiones de arquitectura y procedimientos de despliegue de forma que un tercero pueda reproducir el entorno sin asistencia del autor.
- **Ética Profesional:** Garantizar la originalidad del proyecto y el uso responsable de herramientas de inteligencia artificial.

4. Resultado del Aprendizaje

4.1. Objetivo SMART

Específico (¿Qué?)	Medible (¿Cuánto?)	Alcanzable (¿Cómo?)	Realista (¿Para qué?)	A Tiempo (¿Cuándo?)
Que el estudiante domine el empaquetado, la parametrización y el ciclo de vida de aplicaciones distribuidas en Kubernetes mediante Helm.	Desplegar 4 microservicios, 1 broker de mensajería, 1 base de datos persistente y 2 cronjobs desde un único chart, con al menos 2 archivos de valores por ambiente y 1 rollback comprobado.	Utilizando Docker, un clúster local de Kubernetes (minikube / kind / k3s), Helm 3, un Ingress Controller y un broker de mensajería desplegado como dependencia del chart.	Fortalecer competencias en arquitectura distribuida, gestión de configuración y preparación para entornos productivos y de nube.	Antes del XX de XXXX de 2026 a las 23:59 hrs, entregando el repositorio vía UEDI.

5. Enunciado de la Práctica.

3.1 Descripción del problema a resolver

Partiendo de los microservicios construidos en la Práctica 4, se le solicita llevar la solución a un nivel de madurez operativa superior. Ya no basta con que los objetos de Kubernetes funcionen: ahora la plataforma completa deberá ser instalable, actualizable y reversible con un solo comando, además de incorporar comunicación asíncrona, persistencia durable y controles de seguridad.

El namespace de trabajo será **sa-p5** y deberá ser creado por el propio chart, no de forma manual. Queda prohibido aplicar manifiestos sueltos con *kubectl apply -f*: el único mecanismo de despliegue aceptado será *helm install / helm upgrade*.

A. Empaquetado con Helm (obligatorio)

- Deberá construir un **chart padre** (por ejemplo, *sa-platform*) que contenga como **subcharts** cada uno de los microservicios y el simulador de API Gateway.
- El broker de mensajería y la base de datos deberán declararse como **dependencias** en el archivo *Chart.yaml* (por ejemplo, desde el repositorio de Bitnami) y resolverse con *helm dependency update*.

- Deberá existir un *values.yaml* base más, como mínimo, *values-dev.yaml* y *values-prod.yaml*, que modifiquen al menos: número de réplicas, límites de recursos, tag de la imagen y nivel de log.
- Las plantillas deberán demostrar uso real del motor de plantillas: un archivo *_helpers.tpl* con al menos dos *named templates*, y el uso de *range*, *if / else*, *required*, *default* y *quote*. No se aceptarán plantillas que sean manifiestos estáticos disfrazados.
- *helm lint* deberá ejecutarse sin errores ni advertencias.
- El chart deberá estar versionado (*version* y *appVersion* en *Chart.yaml*). Deberá demostrar al menos **dos versiones publicadas**, una operación de *helm upgrade* y una de *helm rollback* a la revisión anterior, evidenciadas con *helm history*.

B. Configuración y manejo de secretos (obligatorio)

- Toda variable no sensible deberá provenir de un **ConfigMap** generado por el chart.
- Las credenciales de la base de datos y del broker deberán almacenarse en un **Secret** e inyectarse mediante *envFrom* o volumen montado. **Ninguna credencial podrá estar escrita en el repositorio**; se entregará un *values.example.yaml* con valores ficticios.
- Un cambio en el ConfigMap deberá provocar el reinicio automático de los pods afectados (sugerencia: anotación con *checksum/config*).

C. Persistencia de datos (obligatorio)

- La base de datos deberá desplegarse dentro del clúster como **StatefulSet** con su **PersistentVolumeClaim** y un *headless service* asociado.
- Deberá demostrar, con evidencia, que los datos **sobreviven al borrado del pod** de la base de datos.

D. Comunicación asíncrona mediante broker (obligatorio)

- Deberá incorporar un **broker de mensajería** desplegado en el clúster: RabbitMQ, Kafka o NATS.
- Al menos **un flujo de negocio** deberá dejar de ser síncrono: un productor publica el evento y retorna inmediatamente, y un microservicio consumidor lo procesa de forma independiente.
- La cola o tópico deberá ser **durable**, y el consumidor deberá confirmar el mensaje solo tras procesarlo correctamente.
- Deberá demostrar el comportamiento del sistema cuando el consumidor está caído: los mensajes se acumulan y se procesan al restaurarlo, sin pérdida de información.

E. Exposición y aislamiento de red (obligatorio)

- La única puerta de entrada al clúster será el **API Gateway**, expuesto mediante un **Ingress** con un Ingress Controller (NGINX o Traefik) y enrutamiento por *path* o por *host*.
- Los microservicios, la base de datos y el broker no podrán exponerse con *NodePort* ni *LoadBalancer*.

- Deberá definir **NetworkPolicies** que impidan el tráfico lateral: solo el gateway puede alcanzar a los microservicios, y solo los microservicios autorizados pueden alcanzar a la base de datos y al broker. Deberá entregar evidencia del bloqueo (una petición que falle desde un pod no autorizado).

F. Salud, escalado y resiliencia (obligatorio)

- Cada deployment deberá definir **liveness**, **readiness** y **startup probe**, con valores justificados en la documentación.
- Deberá configurar **HPA** con mínimo 2 y máximo 5 réplicas, activándose al 70 % de uso de CPU, con *metrics-server* habilitado.
- El namespace deberá contar con **ResourceQuota** y **LimitRange** que acoten el consumo total.
- Cada microservicio deberá tener un **PodDisruptionBudget**.
- La estrategia de actualización deberá ser *RollingUpdate* con *maxUnavailable: 0*, y deberá demostrar una actualización **sin caída de servicio** (peticiones continuas durante el upgrade, sin respuestas de error).

G. Seguridad del contenedor y del clúster (obligatorio)

- Cada cronjob y cada microservicio deberá ejecutarse con un **ServiceAccount** dedicado, con **Role** y **RoleBinding** de mínimo privilegio. Queda prohibido el uso del *ServiceAccount default*.
- Los contenedores deberán declarar **securityContext** con *runAsNonRoot: true*, *readOnlyRootFilesystem: true* y *allowPrivilegeEscalation: false*.
- Las imágenes deberán construirse con **multi-stage build** y una base mínima (alpine, slim o distroless). Deberá entregar una tabla comparativa del **tamaño de cada imagen antes y después** de la optimización.

H. Trabajos programados (obligatorio)

- **Cronjob 1:** se ejecuta cada 2 minutos e inserta en la base de datos un registro con la fecha y hora de ejecución (zona horaria GMT-6) y el número de carné del estudiante.
- **Cronjob 2:** se ejecuta cada 10 minutos, consulta los registros generados por el Cronjob 1, calcula un resumen (cantidad de ejecuciones por hora) y publica ese resumen como un mensaje en el broker, donde deberá ser consumido y almacenado.
- Ambos deberán definir *concurrencyPolicy: Forbid*, *backoffLimit*, *successfulJobsHistoryLimit* y *failedJobsHistoryLimit*.

I. Pruebas de carga (obligatorio)

- Deberá elaborar un script de carga con **k6**, **Locust** o **hey** que golpee el API Gateway con concurrencia creciente.
- Deberá evidenciar el **escalado automático** (salida de *kubectl get hpa -w* y *kubectl get pods -w*) y el posterior **descenso de réplicas** al cesar la carga.
- Reporte, como mínimo: peticiones por segundo, latencia p95 y porcentaje de error.

3.2 Alcance de la práctica

Obligatorio:

- **Chart de Helm único, parametrizado, versionado y con dependencias declaradas.**
- **Despliegue de los 4 microservicios más el API Gateway en el namespace sa-p5.**
- **ConfigMaps, Secrets y persistencia con StatefulSet + PVC.**
- **Broker de mensajería con al menos un flujo asíncrono funcional.**
- **Ingress como único punto de entrada y NetworkPolicies de aislamiento.**
- **Probes, HPA, ResourceQuota, LimitRange y PodDisruptionBudget.**
- **RBAC de mínimo privilegio y securityContext restrictivo.**
- **Dos cronjobs encadenados y pruebas de carga documentadas.**

No requerido (pero recomendado):

- **Uso de Redis como caché de lectura para uno de los microservicios.**
- **Publicación del chart en un repositorio de charts propio (GitHub Pages u OCI).**
- **Uso de helm test para validar el release tras la instalación.**
- **Definición de affinity o topologySpreadConstraints para distribuir réplicas.**

Fuera de alcance (se abordará en prácticas posteriores):

- **Despliegue en clúster de nube administrado e infraestructura como código.**
- **Observabilidad (métricas, trazas y logs centralizados) y service mesh.**
- **Integración y entrega continua.**

3.3 Requerimientos técnicos

El estudiante deberá utilizar o integrar las siguientes herramientas y tecnologías:

- **Orquestación y contenedores:** Docker, Kubernetes (minikube / kind / k3s).
- **Gestión de paquetes:** Helm 3 (chart padre con subcharts y dependencias).
- **Mensajería:** RabbitMQ, Kafka o NATS.
- **Persistencia:** Base de datos relacional o no relacional desplegada como StatefulSet.
- **Exposición:** Ingress Controller (NGINX o Traefik).
- **Tareas programadas:** CronJobs de Kubernetes.
- **Pruebas de carga:** k6, Locust o hey.
- **Lenguajes de programación:** Node.js, Java, Python o Go (mínimo dos distintos, heredados de la Práctica 4).
- **Control de versiones:** Repositorio en la nube (GitHub, GitLab o Bitbucket).

6. Entregables

Tipo	Descripción
Repositorio en la Nube	GitHub, GitLab o similar. Nombre: Practicas-SA-<<SECCIÓN>>-<<CARNE>>
Código Fuente	Carpeta /P5 con la implementación completa, organizada y comentada según buenas prácticas.
Archivos Docker	Un Dockerfile multi-stage por cada microservicio, más el .dockerignore correspondiente.
Chart de Helm	Carpeta /P5/charts con el chart padre, subcharts, Chart.yaml con dependencias, values.yaml, values-dev.yaml, values-prod.yaml, values.example.yaml y _helpers.tpl.
Scripts	Script de carga (k6 / Locust / hey) y scripts de los dos cronjobs.
Documentación	1. Diagrama de arquitectura, indicando elementos internos y externos al clúster, flujos síncronos y asíncronos, y los límites impuestos por las NetworkPolicies. 2. Detalle reproducible de todos los comandos utilizados, desde el clúster vacío hasta la aplicación funcionando. 3. Tabla comparativa del tamaño de las imágenes antes y después de la optimización. 4. Evidencias de: helm history con rollback, escalado por HPA bajo carga, persistencia tras el borrado del pod de base de datos, bloqueo por NetworkPolicy y actualización sin downtime. 5. Resultados de la prueba de carga (RPS, latencia p95 y tasa de error).

	6. Preguntas teóricas: ¿Qué es Helm y qué problema resuelve frente a los manifiestos sueltos?, ¿diferencia entre chart, release y repository?, ¿qué es un StatefulSet y cuándo NO usarlo?, ¿diferencia entre liveness, readiness y startup probe?, ¿qué es una NetworkPolicy y por qué el tráfico es permitido por defecto?, ¿qué es un PodDisruptionBudget?, ¿qué ventajas y qué nuevos problemas introduce la comunicación asíncrona?, ¿qué hace helm rollback internamente?
--	---

7. Material de apoyo

- [Documentación oficial de Kubernetes](#)
- [Documentación oficial de Helm](#)
- [Helm - Chart Template Guide](#)
- [Kubernetes - Network Policies](#)
- [Kubernetes - StatefulSets](#)
- [Kubernetes - Liveness, Readiness y Startup Probes](#)
- [Documentación oficial de Docker](#)
- [Documentación de k6](#)
- [Martin Fowler - Microservices Architecture](#)

8. Recursos y herramientas a utilizar

- **Software/Hardware:** Docker Desktop o Docker Engine, Kubernetes local (minikube / kind / k3s) con al menos 4 GB de RAM asignados, Helm 3, kubectl, metrics-server e Ingress Controller.
- **Lenguajes:** Node.js / Java / Python / Go (Mínimo 2 distintos).
- **Plataformas:** GitHub / GitLab / Bitbucket (para repositorio).

9. Cronograma

Tipo	Fecha Inicio	Fecha Fin
Asignación de Práctica	20/08/2026	20/08/2026
Elaboración	20/08/2026	27/08/2026
Calificación	29/08/2026	29/08/2026

10. Rúbrica de Calificación

8.1 Requisitos para optar a la calificación

Antes de la evaluación de la práctica, los estudiantes deben cumplir con los requisitos que se indiquen en esta sección.

Tema	Descripción	Cumple (Sí/No)
Despliegue con Helm	La totalidad de la plataforma se instala con helm install / helm upgrade. No se aplicaron manifiestos sueltos.	
Práctica 4	Haber cumplido con la entrega y calificación de la práctica número 4.	
Repositorio	Cumple con la estructura de carpetas y nombrado solicitada.	
Credenciales	No existen credenciales reales versionadas en el repositorio.	
Documentación	Documentación entregada, completa y organizada, con evidencias (Sin IA).	

8.2 Resumen de Puntuaciones

Área	Puntos Totales (Base 100)	Puntos Obtenidos
1. Habilidades (40%)		

Documentación técnica y diagrama de arquitectura	12	
Calidad y estructura del chart de Helm	12	
Organización del repositorio	3	
Lista de comandos reproducibles	3	
Preguntas teóricas	10	
Sub-Total Habilidades	40	
2. Conocimiento (60%)		
Ciclo de vida con Helm (install, upgrade y rollback)	12	
Configuración, secretos y persistencia	10	
Comunicación asíncrona mediante broker	12	
Exposición, aislamiento de red y seguridad	10	
Escalado y resiliencia bajo carga	10	
Cronjobs encadenados y funcionales	6	

Sub-Total Conocimiento	60	
TOTAL (Escalable a 5.00 pts)	100	

Detalle de la Calificación

No.	Criterio de evaluación	Punteo máximo	Satisfactorio (100% - 61%)	Necesita mejorar (60% - 0%)	Puntuación Obtenida
1	Habilidades	40			
1.1	Documentación técnica y Arquitectura	12	Diagrama claro y completo: distingue elementos internos y externos, flujos síncronos y asíncronos, y los límites que imponen las NetworkPolicies. Incluye todas las evidencias solicitadas.	Diagrama confuso, incompleto o sin distinguir el flujo asíncrono. Faltan evidencias.	
1.2	Calidad y estructura del chart de Helm	12	Chart padre con subcharts y dependencias declaradas, valores por ambiente, _helpers.tpl con named templates y uso real de range, if, required y default. helm lint sin observaciones.	Plantillas que son manifiestos estáticos con otro nombre, valores quemados, ausencia de subcharts o errores en helm lint.	
1.3	Organización del Repositorio	3	Uso correcto de /P5, nombrado correcto de repo, separación clara entre código, charts y documentación.	Carpetas desordenadas o mal nombradas.	
1.4	Lista de comandos reproducibles	3	Lista detallada que permite levantar el entorno completo desde cero sin conocimiento previo del proyecto.	Comandos incompletos, en desorden o erróneos.	
1.5	Preguntas Teóricas	10	Respuestas con análisis propio, directas, claras y correctas técnicamente.	Respuestas copiadas de internet/IA, incompletas o técnicamente incorrectas.	

2	Conocimiento	60			
2.1	Ciclo de vida con Helm	12	La plataforma completa se instala con un solo comando. Se demuestran dos versiones del chart, un upgrade exitoso y un rollback verificado con helm history.	Requiere pasos manuales, no hay versionado del chart o el rollback no funciona.	
2.2	Configuración, secretos y persistencia	10	ConfigMaps y Secrets correctamente generados e inyectados, sin credenciales versionadas. StatefulSet con PVC y evidencia de que los datos sobreviven al borrado del pod.	Valores quemados en las plantillas, credenciales expuestas o pérdida de datos al reiniciar el pod.	
2.3	Comunicación asíncrona	12	Broker desplegado como dependencia del chart. El flujo asíncrono funciona de extremo a extremo, la cola es durable y no se pierden mensajes con el consumidor caído.	El broker está desplegado pero no se usa realmente, el flujo sigue siendo síncrono o se pierden mensajes.	
2.4	Exposición, red y seguridad	10	Solo el API Gateway es accesible mediante Ingress. Las NetworkPolicies bloquean el tráfico lateral con evidencia. RBAC de mínimo privilegio y securityContext restrictivo aplicados.	Servicios internos expuestos, ausencia de NetworkPolicies, uso del ServiceAccount default o contenedores ejecutándose como root.	
2.5	Escalado y resiliencia	10	Probes correctamente diferenciadas, HPA escalando de forma comprobable bajo carga, cuotas y PDB aplicados, y actualización sin downtime demostrada.	Probes copiadas sin criterio, el HPA no escala, no hay evidencia de la prueba de carga o el servicio cae durante el upgrade.	

2.6	Cronjobs encadenados	6	El primer cronjob inserta cada 2 minutos la fecha GMT-6 y el carné. El segundo agrega la información cada 10 minutos y la publica en el broker, donde es consumida.	Falla la conexión, la expresión cron, el formato de fecha o el segundo cronjob no se comunica con el broker.	
-----	----------------------	---	---	--	--